Guía para Deploy: Render + Neon

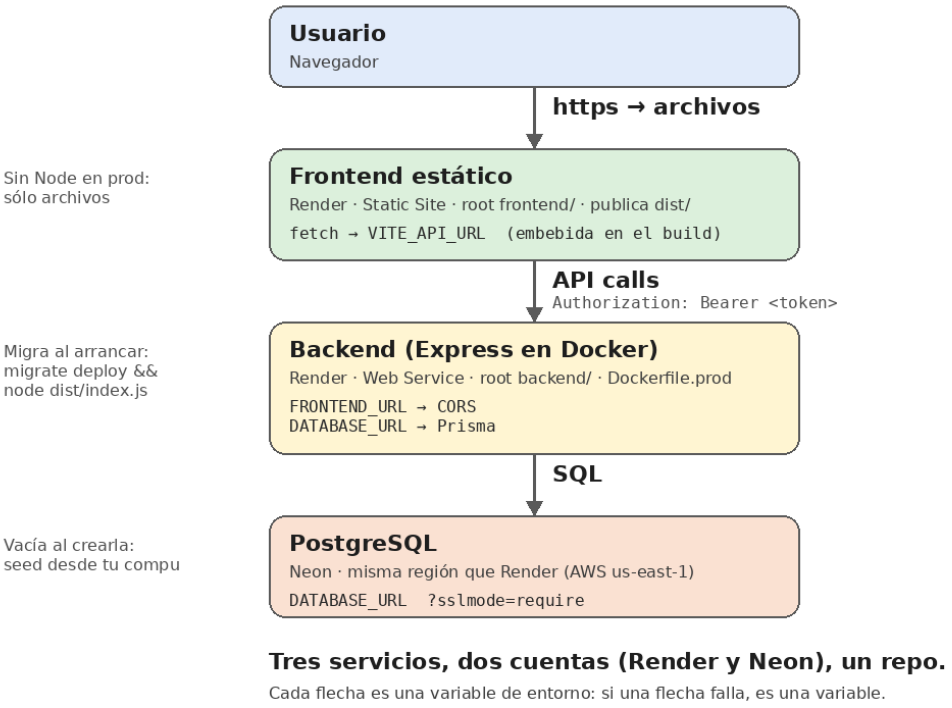
Procedimiento paso a paso para poner la app en producción, gratis y sin tarjeta.
Acompaña la Clase 23 y las actividades [AG-C23] y [AI-C23].

Sobre este documento

Esta guía es un **procedimiento**, no teoría: la teoría está en las filminas de la Clase 23. La idea es que la sigan con la compu adelante, en orden, sin saltear pasos. Cada paso dice qué hacer, dónde, y qué tenés que llevarte de ahí para el paso siguiente.

Sirve tanto para el **TP grupal** (con la cuenta del grupo) como para la **Actividad individual** (con tu cuenta personal). Las diferencias para la individual están en la sección correspondiente al final.

Deployar es correr lo mismo que corrés en tu compu, en una compu ajena, con variables distintas. Si algo no anda, casi siempre es una variable. Lo que vas a tener al final:



Tres servicios, dos cuentas (Render y Neon), un repo.
Cada flecha es una variable de entorno: si una flecha falla, es una variable.

Convenciones

- `{{ASÍ}}` es un valor que tenés que reemplazar por el tuyo. Nunca lo pegues literal.
- Los recuadros amarillos 📸 **CAPTURA** marcan pantallas que se agregan a la guía. Donde ya hay una imagen, es la salida real del repo de ejemplo.

- Nombres de campos de Render y Neon al 10/2026. Si un campo cambió de nombre, buscá el concepto: "carpeta raíz", "comando de build", "variables de entorno".
- Referencia de código: repo de ejemplo `DS26-Libreria`, branch `C23`. Los archivos nuevos están completos en el anexo.

Índice

Paso 0 — Antes de tocar ninguna plataforma (sin internet)
Paso 1 — Neon: la base de datos
Paso 2 — Render: el backend (Web Service con Docker)
Paso 3 — Render: el frontend (Static Site)
Paso 4 — Cerrar el círculo: CORS
Paso 5 — Datos de demo: el seed
Paso 6 — Verificar y avisar
Cómo se mantiene solo (y qué cuesta la capa gratuita)
Qué falla y por qué
Para la Actividad individual
Fuera de esta guía
Anexo — Archivos completos

Paso 0 — Antes de tocar ninguna plataforma (sin internet)

Esto es la mitad del deploy y se hace en tu compu. Si llegás a la clase con esto hecho, el resto es apretar botones.

0.1 Backend

☐ `backend/Dockerfile.prod` y `backend/tsconfig.build.json` existen (copialos del anexo o del branch `C23`).

☐ `package.json` tiene `"build": "tsc -p tsconfig.build.json"` y `"start": "node dist/index.js"`.

☐ `npm run build` compila limpio. Si tu back corre en Docker: `docker compose run --rm api npm run build`.

☐ `PORT` y `FRONTEND_URL` se leen de `process.env`, con un valor por defecto para local.

☐ `prisma/migrations/` está commiteado y sin pendientes: `docker compose run --rm api npx prisma migrate dev` no pide crear nada.

☐ `backend/.env.example` tiene todas las variables y ningún valor real.

☐ `src/generated` está en `.gitignore` (se regenera en el build).

```
// backend/src/index.ts
const PORT = Number(process.env.PORT) || 3000;

// backend/src/app.ts
const corsOptions = { origin: [process.env.FRONTEND_URL ?? "http://localhost:5173"] };
```

Así se ve un build limpio del back (salida real del ejemplo): compila, `dist/` tiene lo mismo que `src/` y ningún test adentro.

backend — npm run build

```
$ npm run build

> libreria-api@1.0.0 build
> tsc -p tsconfig.build.json

$ ls dist
app.js
config
controllers
generated
index.js
middlewares
routes
services
types
validations
$ find dist -name "*.test.js" | wc -l
0
```

0.2 Frontend

- ☐ `npm run build` compila limpio, con los tests adentro.
- ☐ A la API se llega **sólo** por `import.meta.env.VITE_API_URL`. Este comando tiene que devolver vacío: `grep -rn "localhost:3000" src`.
- ☐ Rutas protegidas funcionando: `/libros/nuevo` sin login te manda a `/login`.
- ☐ `frontend/.env.example` tiene `VITE_API_URL`.

frontend — npm run build

```
$ cd frontend
$ VITE_API_URL=https://libreria-api.onrender.com/api npm run build
> libreriaapp@0.0.0 build
> tsc -b && vite build
vite v8.0.3 building client environment for production...
[2K
transforming...✓ 438 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                          0.46 kB | gzip:   0.31 kB
dist/assets/libroIcono-BjXgBxFP.png  121.84 kB
dist/assets/index-AbXnwv5E.css       233.86 kB | gzip:  31.88 kB
dist/assets/index-DPrTIMna.js        382.55 kB | gzip: 119.60 kB
✓ built in 1.78s
```

Y la prueba de que la variable queda **embebida** en el JS (por eso nunca va un secreto en el front):

```
frontend — la variable queda en el bundle

$ grep -o "https://[a-z.-]*onrender.com/api" dist/assets/index-*.js
https://libreria-api.onrender.com/api

# ← la URL quedó literal en el JS que baja el navegador
```

0.3 Repo y cuentas

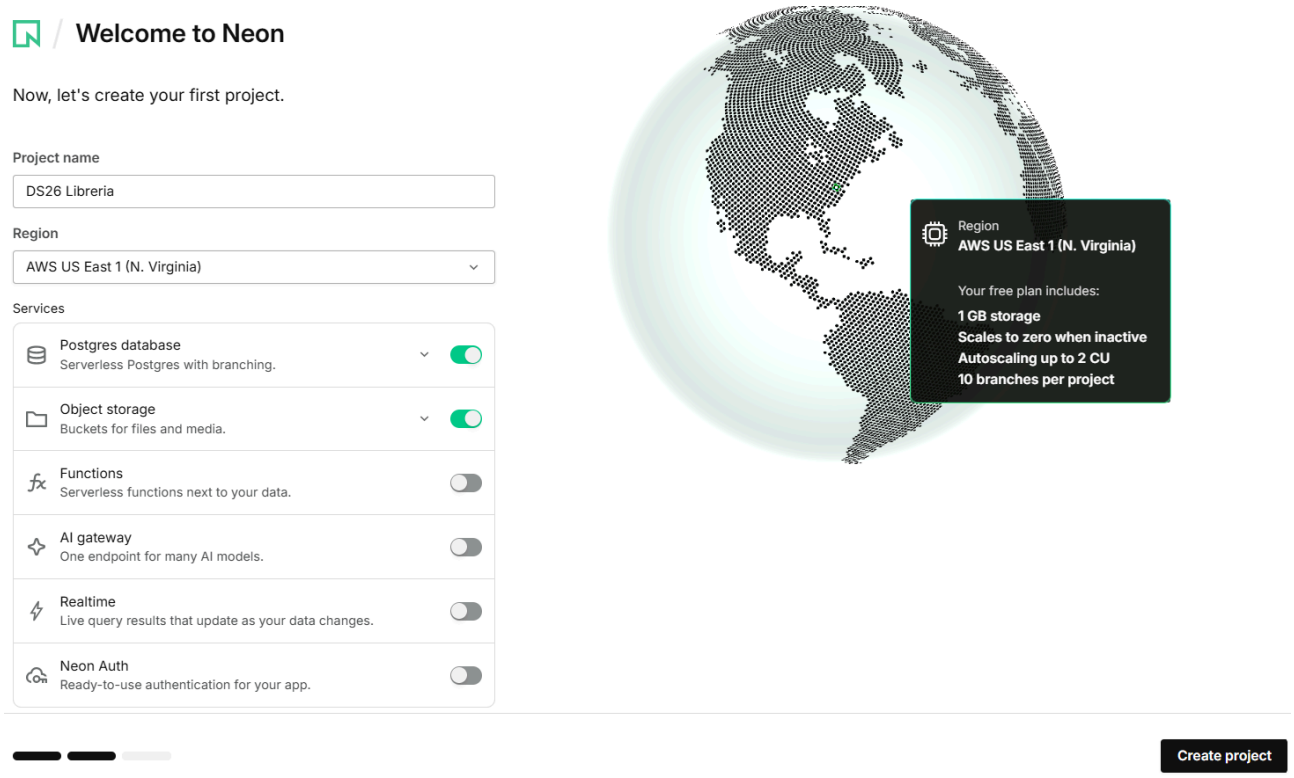
☐ Nada de `.env`, `dist/` ni `node_modules/` commiteados. Todo lo de arriba está en `main`.

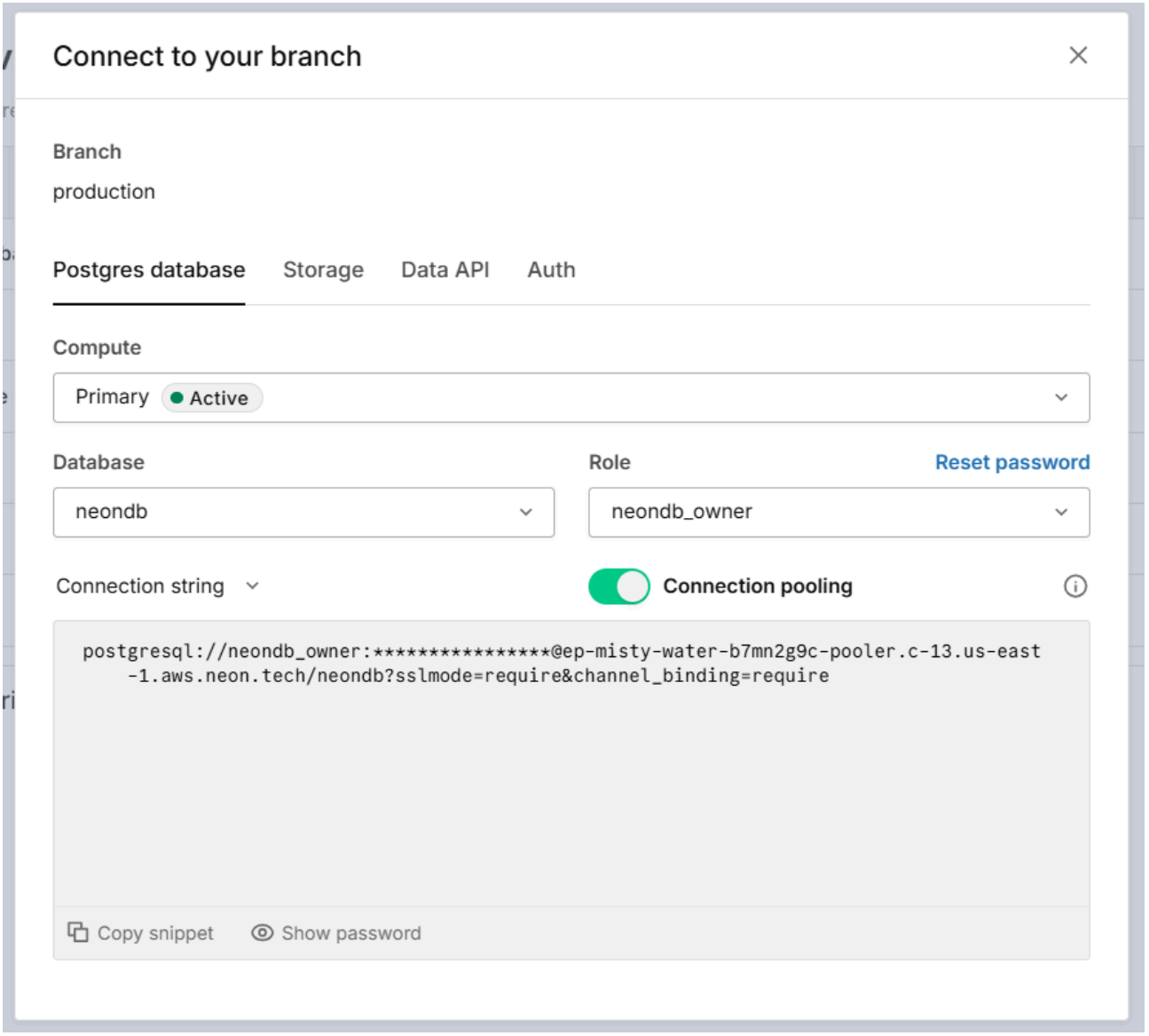
☐ Un **mail del grupo**. Con ese mail: cuenta en render.com (plan Hobby) y en neon.tech (plan Free). Ninguna pide tarjeta. No creen nada todavía: sólo la cuenta.

☐ La cuenta de GitHub que haga el deploy tiene acceso al repo del grupo (Render lo va a pedir).

Paso 1 — Neon: la base de datos

1. Entrá a neon.tech con el mail del grupo y creá un **proyecto nuevo**.
2. Región: **AWS · US East (N. Virginia) · us-east-1**. Es la misma región donde van a estar los servicios de Render. Si quedan en regiones distintas, cada consulta cruza medio continente.
3. Abrí el panel de conexión del proyecto y elegí la **connection string directa** (no la "pooled"). Tiene esta forma:
   `postgresql://{{usuario}}:{{password}}@ep-{{algo}}.us-east-1.aws.neon.tech/neondb?sslmode=require...`
4. Copiala a un lugar seguro del grupo (no a Discord, no al README). Es tu `DATABASE_URL` y contiene la contraseña.


Welcome to Neon
Now, let's create your first project.
Project name
DS26 Libreria
Region
AWS US East 1 (N. Virginia)
Services
Postgres database
Serverless Postgres with branching.
Object storage
Buckets for files and media.
Functions
Serverless functions next to your data.
AI gateway
One endpoint for many AI models.
Realtime
Live query results that update as your data changes.
Neon Auth
Ready-to-use authentication for your app.
Region
AWS US East 1 (N. Virginia)
Your free plan includes:
1 GB storage
Scales to zero when inactive
Autoscaling up to 2 CU
10 branches per project
Create project



Nota

La URL que te da Neon puede terminar en `?sslmode=require&channel_binding=require`. Pegala tal cual: funciona.

No hace falta `DIRECT_URL` ni `pgbouncer=true`: con `@prisma/adapter-pg` el back se conecta directo.

La base nueva está **vacía**. Las tablas las crean las migraciones cuando arranque el back (paso 2); los datos los cargás vos (paso 5).

Plan Free de Neon: 100 CU-hora de cómputo por proyecto por mes y 1 GB de almacenamiento. El cómputo se apaga solo a los 5 minutos sin uso y vuelve en una fracción de segundo. Nada se borra si tocás un límite.

Paso 2 — Render: el backend (Web Service con Docker)

1. En el dashboard de Render: **New** → **Web Service**. Conectá tu cuenta de GitHub y elegí el repo del grupo.
2. Completá la configuración con la tabla de abajo. Lo que no está en la tabla queda como viene.
3. Cargá las **variables de entorno** antes del primer deploy (segunda tabla).
4. Creá el servicio y mirá el log del deploy.
5. Cuando diga que está live, abrí la URL que te dio Render: `https://{{tu-back}}.onrender.com/` tiene que responder el JSON de bienvenida de tu API. **Copiá esa URL**: la necesitás en el paso 3.

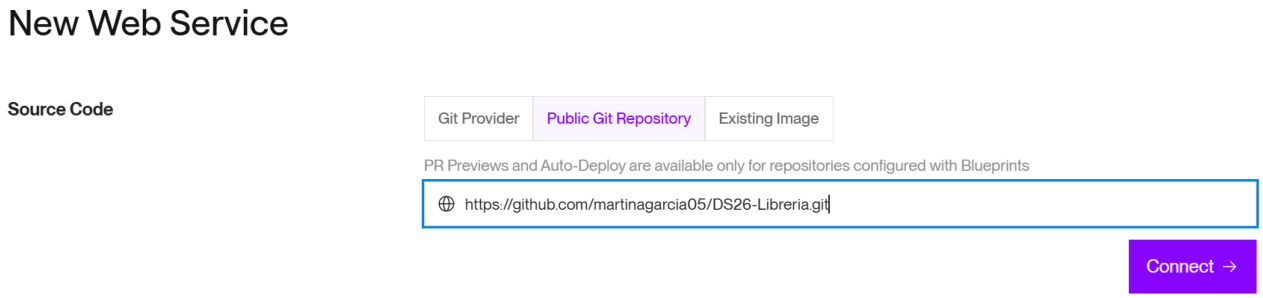
Configuración del servicio

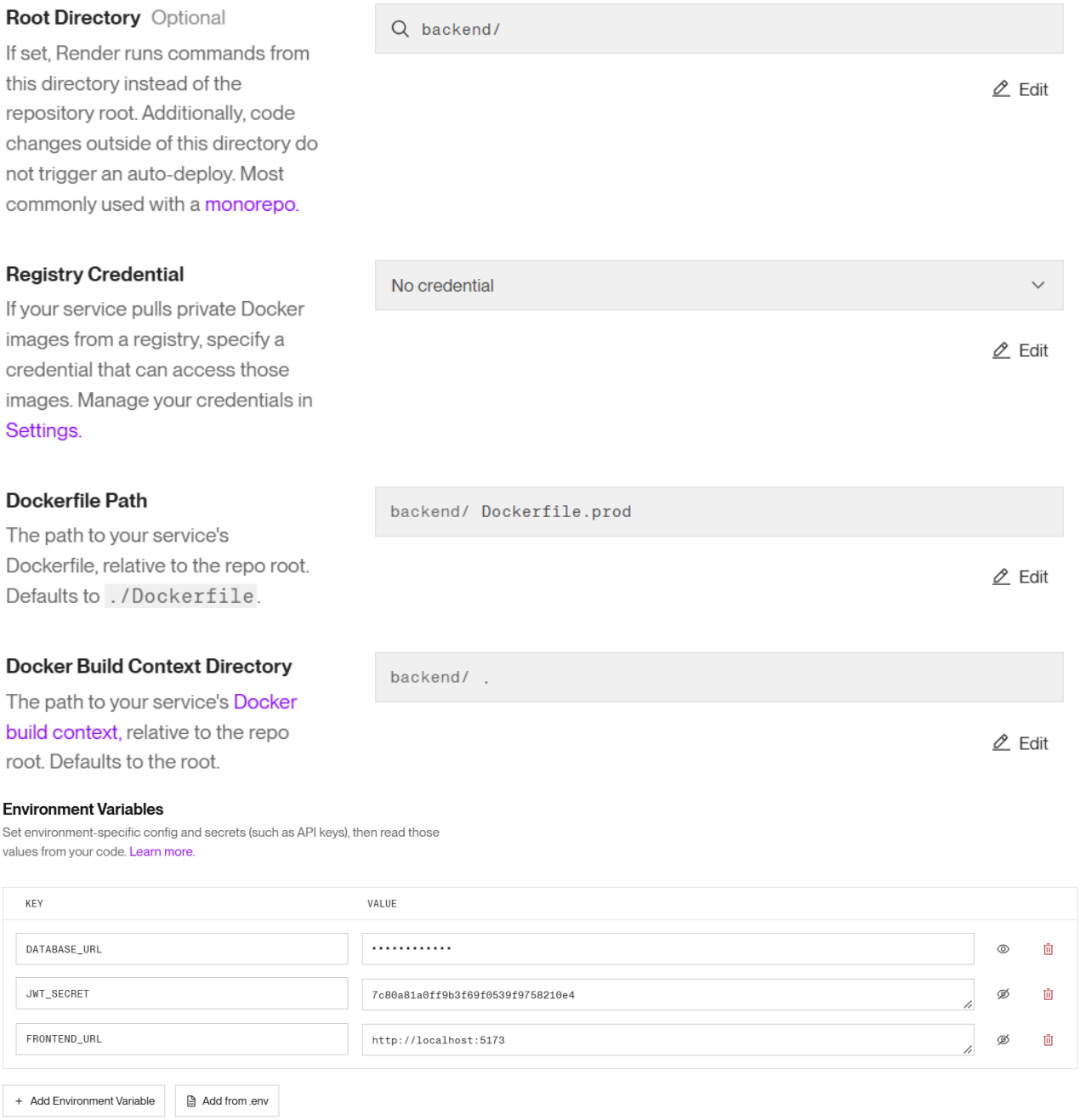
Campo	Valor
Language / Runtime	**Docker**
Branch	`main`
Root Directory	`backend`
Dockerfile Path	`./Dockerfile.prod` (relativo a `backend/`)
Region	US East (la misma que Neon)
Instance type / Plan	**Free**
Auto-Deploy	On Commit (es el default: dejalo)

Variables de entorno

Variable	Valor	Por qué
`DATABASE_URL`	la connection string de Neon (paso 1)	Prisma la usa para migrar y para consultar
`JWT_SECRET`	usá el botón **Generate** de Render	un valor aleatorio que nadie tipea ni pega en ningún lado
`FRONTEND_URL`	`http://localhost:5173` **por ahora**	el front todavía no existe; se corrige en el paso 4

`PORT` **no se carga**: Render la inyecta sola y Express la lee de `process.env.PORT`.





Qué tiene que verse en el log

- El build de la imagen en dos etapas (`build` y la final), con `npm ci`, `prisma generate` y `npm run build`.
- `prisma migrate deploy` aplicando tus migraciones (la primera vez, todas; después, "No pending migrations to apply").

- `Servidor escuchando en http://localhost:10000` (o el puerto que Render haya elegido).

```
02:04:49 PM  [qcpwm]  All migrations have been successfully applied.
02:04:52 PM  [qcpwm]  Servidor escuchando en http://localhost:10000
02:04:58 PM           ==> Your service is live 🎉
02:04:59 PM           ==>
02:04:59 PM           ==> ///////////////////////////////////////////////////////
02:04:59 PM           ==>
02:04:59 PM           ==> Available at your primary URL https://ds26-libreria.onrender.com
02:04:59 PM           ==>
02:04:59 PM           ==> ///////////////////////////////////////////////////////
```

> **Nota**
> **Pull Request Previews** en el back: dejalo **apagado**. Un preview correría tus migraciones contra la base real.
> Si el deploy falló, andá a la sección "Qué falla y por qué". El 90% de las veces es una de las tres variables.

Paso 3 — Render: el frontend (Static Site)

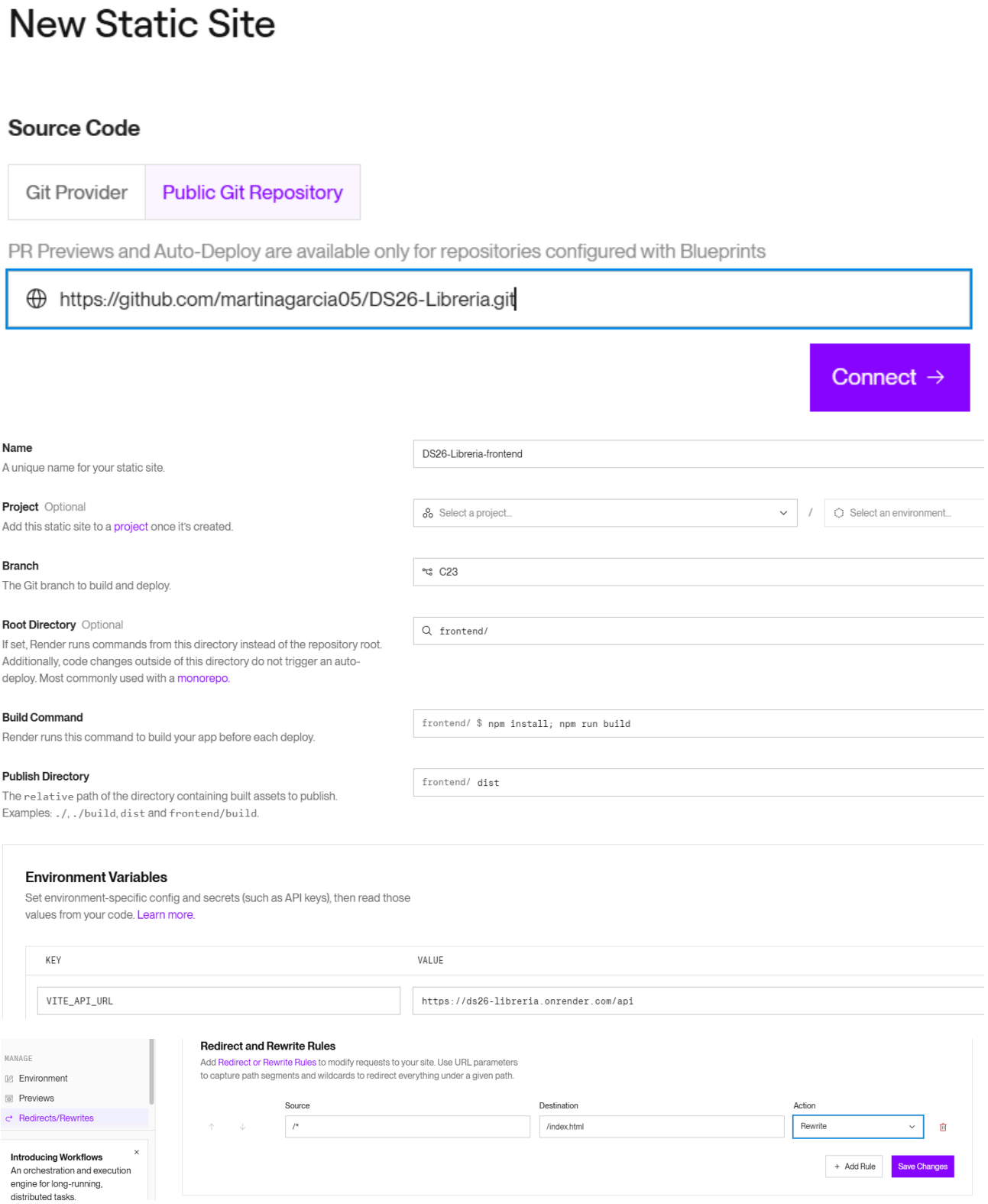
1. **New** → **Static Site**, mismo repo.
2. Configuración con la tabla de abajo.
3. Cargá la variable `VITE_API_URL`. Es una variable de **build**: queda pegada dentro del JS, por eso el servicio tiene que rebuildear si la cambiás.
4. Agregá la regla de **Redirects / Rewrites** para que React Router funcione al refrescar.
5. Creá el sitio. Cuando esté live, abrí la URL. La home carga; el catálogo todavía puede fallar por CORS: es normal, se arregla en el paso 4. **Copiá la URL del front.**

Campo	Valor
Branch	`main`
Root Directory	`frontend`
Build Command	`npm run build`
Publish Directory	`dist`

Variable	Valor
`VITE_API_URL`	`https://{{tu-back}}.onrender.com/api` ← con **/api** al final y sin barra después

Source	Destination	Action
`/*`	`/index.html`	Rewrite

Rewrite ≠ redirect: el servidor devuelve siempre `index.html` y la URL del navegador no cambia; React Router mira la URL y decide qué mostrar.



Nota

Pull Request Previews en el Static Site podés dejarlos activados si te sirven: no tocan la base.

Paso 4 — Cerrar el círculo: CORS

Volvé al **Web Service del back** → **Environment** y cambiá:

Variable	Valor
FRONTEND_URL	https://{{tu-front}}.onrender.com ← **sin barra al final**

Guardar **redeploya el back solo**: cambiar una variable de entorno es un deploy. Cuando termine, el catálogo carga y la consola del navegador no muestra errores de CORS.

Por qué este orden: cada URL existe recién cuando el servicio existe. El back necesita la del front (CORS) y el front la del back (fetch), así que uno de los dos arranca con un valor provisorio y se corrige acá.



Paso 5 — Datos de demo: el seed

Las migraciones crearon las tablas; la base sigue **vacía**, y sin usuarios no hay login. El seed se corre **una sola vez**, desde tu compu, apuntando a Neon:

```
cd backend
docker compose run --rm -e DATABASE_URL="{{DATABASE_URL de Neon}}" api npx prisma db seed
# → Seed listo
```

- Usa tu imagen de **desarrollo** (la del `docker-compose.yml`), que sí tiene `tsx` y `prisma/seed.ts`. La de producción no los tiene.
- `-e DATABASE_URL=…` pisa la del `.env` **sólo para ese comando**. Tu base local no se toca.
- El seed del ejemplo es idempotente (`upsert`, `skipDuplicates`): correrlo dos veces no duplica nada. Si el tuyo no lo es, hacelo idempotente antes.
- Si tu grupo no tiene seed, es el momento de escribir uno: usuarios de prueba por rol y los datos mínimos para la demo. No cargues datos reales del cliente.

Nota
Por qué desde tu compu y no desde Render: el plan Free no tiene shell ni jobs sueltos, y la imagen de producción no lleva `src/`. Es una deuda técnica declarada: en un equipo real los datos iniciales son un script versionado del pipeline.

Paso 6 — Verificar y avisar

- ☐ Abrir el front en la URL pública y loguearse con un usuario del seed.
- ☐ Crear, editar y borrar algo: un CRUD completo contra Neon.
- ☐ F5 en una ruta interna (por ejemplo `/catalogo`): no da 404.
- ☐ Consola del navegador (F12) sin errores de CORS ni de `fetch`.
- ☐ Entrar a `/libros/nuevo` (o su equivalente) sin login: redirige a `/login`.
- ☐ Las dos URLs + usuarios de prueba en el canal del grupo en Discord, hilo `[AG-C23]`. Antes del **jueves previo** a la presentación de tu grupo.

Cómo se mantiene solo (y qué cuesta la capa gratuita)

Auto-deploy

- `git push` a `main` → Render detecta el commit → cada servicio reconstruye **sólo si cambió algo dentro de su Root Directory** (`frontend/` o `backend/`). Un cambio sólo en el front no reconstruye el back.
- Si el build o las migraciones fallan, el deploy falla y **sigue corriendo la versión anterior**. Lo ves en rojo en el dashboard y el log dice por qué.
- Cambiar una variable de entorno también redeploya ese servicio.
- Deployá desde `main`. Cada push gasta minutos de build del plan.

El back se duerme

- Un Web Service Free se **apaga a los 15 minutos sin tráfico** y se vuelve a levantar con el próximo request. Según Render, levantarlo lleva **alrededor de un minuto**; mientras tanto el navegador ve una página de carga de Render.
- No es un error. Antes de una demo: **abrí la app y logueate 3 minutos antes** de pasar al frente. El front (Static Site) no se duerme.
- Neon también apaga el cómputo a los 5 minutos sin uso, pero vuelve en milisegundos: no lo vas a notar.

Límites del plan gratuito (al 10/2026)

Plataforma	Límite	Qué pasa si lo pasás
Render (Hobby)	750 horas de instancia por mes, compartidas entre tus Web Services Free (el tiempo dormido no cuenta)	Render suspende tus servicios Free hasta el mes siguiente
Render (Hobby)	500 minutos de build pipeline por mes	sin tarjeta cargada, no hay más builds hasta el mes siguiente; lo deployado sigue corriendo
Render (Hobby)	5 GB de transferencia saliente por mes	sin tarjeta cargada, suspende los servicios Free hasta el mes siguiente

Plataforma	Límite	Qué pasa si lo pasás
Render (Hobby)	25 servicios y un solo miembro por workspace	por eso una cuenta del grupo, no una por integrante
Neon (Free)	100 CU-hora por proyecto por mes · 1 GB de almacenamiento	el cómputo se suspende hasta el mes siguiente; nada se borra

Un solo back que la mayor parte del tiempo está dormido consume una fracción de esos límites. Lo que sí los gasta: pushear cada cinco minutos (minutos de build) y dejar un script pegándole a la API para que no se duerma (horas de instancia).

Prisma

La **misma versión que tenés en el repo**. No corras `npm i prisma@latest` para deployar: cambiar de versión mayor la semana del deploy es la forma más rápida de romper algo que andaba.

Qué falla y por qué

Método: leer el log → cambiar **una sola cosa** → volver a probar. Si te trabás, `#consultas` con el error pegado tal cual (sin la `DATABASE_URL`).

Síntoma	Causa	Arreglo
Consola: `fetch a undefined/libros`	`VITE_API_URL` no estaba cuando se hizo el build (compila igual, sin aviso)	cargar la variable en el Static Site y redeployar
404 en todos los pedidos a la API	`VITE_API_URL` sin `/api` al final	`https://{{back}}.onrender.com/api`
Consola: bloqueado por CORS	`FRONTEND_URL` quedó en `localhost`, o tiene `/` al final	la URL exacta del front, sin barra
Log del back: `Falta JWT_SECRET en el .env`	la variable no está en el panel	agregarla (Generate)
Log del back: `The datasource.url property is required...`	falta `DATABASE_URL`	agregarla
Log del back: `P1001 Can't reach database server at ep-...`	URL mal pegada, o pooled en vez de directa	copiarla de nuevo desde Neon
404 al refrescar en `/catalogo`	falta la regla de rewrite	`/*` → `/index.html`
Build del back falla con `error TS...`	el `tsc` que "casi" andaba en tu compu	arreglarlo local; `npm run build` limpio antes de pushear
Build del front falla	lo mismo	`npm run build` local primero
Log del back: `SECURITY WARNING: The SSL modes...`	aviso informativo de la librería `pg`	no es error; ignorar

Síntoma	Causa	Arreglo
El primer request tarda ~1 minuto	el back estaba dormido	no es error; warm-up antes de la demo
Login da 401 con el usuario del seed	el seed no se corrió contra Neon (o se corrió contra tu base local)	paso 5, con `-e DATABASE_URL`

Los tres errores de variables, tal cual aparecen en el log (verificados sobre el ejemplo):

backend — los tres errores de variables, tal cual salen

```
# Sin JWT_SECRET: el back no arranca
$ node dist/index.js
Error: Falta JWT_SECRET en el .env

# Sin DATABASE_URL: migrate deploy no puede conectarse
$ npx prisma migrate deploy
Loaded Prisma config from prisma.config.ts.
Prisma schema loaded from prisma/schema.prisma.
Error: The datasource.url property is required in your Prisma config file when using
    prisma migrate deploy.

# DATABASE_URL mal pegada o host inalcanzable
PrismaClientKnownRequestError: Can't reach database server at ep-xxx-123.us-
    east-1.aws.neon.tech
  code: 'P1001'
```

Y el aviso que **no** es un error:

backend — el warning de pg que no es un error

```
$ node dist/index.js
Servidor escuchando en http://localhost:10000
(node:521) Warning: SECURITY WARNING: The SSL modes 'prefer', 'require', and 'verify-
    ca'
are treated as aliases for 'verify-full'. ...

# ↑ Esto es un AVISO de la librería pg, no un error. El servidor ya está escuchando.
```

Para la Actividad individual

Mismo procedimiento, con tres diferencias:

1. **Cuenta personal** en Render y en Neon (tu mail), no la del grupo. Es un proyecto más en Neon y dos servicios más en Render: entra de sobra en los límites.
2. **Root Directory**: en `ds2026-actividades` la app no está en la raíz. Si tu copia de C23 está en `ejercicios/c23-deploy/`, el Root Directory del back es `ejercicios/c23-deploy/backend` y el del front `ejercicios/c23-deploy/frontend`. El Dockerfile Path sigue siendo `./Dockerfile.prod` porque es relativo al Root Directory.
3. Render sólo redeploya cuando cambia algo dentro de ese Root Directory: los ejercicios de otras clases no disparan builds.

Opcional: el front en GitHub Pages

Para conocer una segunda plataforma sin arriesgar nada. GitHub Pages sólo sirve archivos estáticos, así que el back sigue en Render. Pistas, no receta:

- En `vite.config.ts`, `base: '/{{nombre-del-repo}}/'`: Pages publica en `usuario.github.io/{{repo}}/`, y sin eso los assets se piden desde la raíz.
- Pages no tiene reglas de rewrite: el truco es copiar `dist/index.html` a `dist/404.html` para que cualquier ruta termine en tu app.
- Publicar `dist/` en un branch `gh-pages` (`npx gh-pages -d dist`) y apuntar Pages a ese branch en Settings → Pages.
- `VITE_API_URL` apuntando a tu back de Render, y en el back `FRONTEND_URL` = la URL de Pages (sin barra final).

Fuera de esta guía

`render.yaml` (infraestructura como código), dominio propio, staging, monitoreo y alertas, pooler de Neon, front dockerizado para desarrollo, GitHub Actions (CI). Todo existe, nada hace falta para la demo. Si alguno los tienta, es una tarde de trabajo cuando lo necesiten.

Anexo — Archivos completos (branch C23)

backend/Dockerfile.prod

```
# Imagen de PRODUCCIÓN del backend. La de desarrollo sigue siendo ./Dockerfile (docker
compose).
# Dos etapas: la primera compila, la segunda se queda sólo con lo que hace falta para
correr.

# ---------- Etapa 1: build ----------
FROM node:22-alpine AS build
WORKDIR /app

# Instala TODO (incluye devDependencies: typescript, prisma CLI). Capa aparte para
cachear.
COPY package*.json ./
RUN npm ci

# El resto del código (.dockerignore deja afuera node_modules, dist, .env, src/generated)
COPY . .

# Genera el cliente de Prisma (no necesita DATABASE_URL) y compila TS → dist/ (sin
tests)
RUN npx prisma generate && npm run build

# ---------- Etapa 2: runtime ----------
FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production

# node_modules viene de la etapa 1: un solo npm ci. El CLI de Prisma queda adentro: migra
al arrancar.
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package*.json prisma.config.ts ./
COPY prisma ./prisma

# Render inyecta PORT; Express lo lee de process.env.PORT (default 3000 para correr
local)
EXPOSE 3000

# 1) aplica las migraciones commiteadas  2) arranca. Si 1) falla, el deploy falla y sigue
la versión anterior.
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/index.js"]
```

backend/tsconfig.build.json

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "outDir": "dist",
    "rootDir": "src",
    "sourceMap": false
  },
```

```
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "src/**/*.test.ts"]
}
```

backend/.dockerignore (líneas agregadas)

```
coverage
src/generated
```

backend/package.json (scripts)

```
"scripts": {
  "dev":   "tsx watch src/index.ts",
  "build": "tsc -p tsconfig.build.json",
  "start": "node dist/index.js",
  "test":  "vitest run",
  "test:watch": "vitest"
}
```

backend/.env.example

```
# Copiá este archivo a .env y completá los valores. El .env NO va a Git.
# En producción (Render) estas variables se cargan en el panel del servicio, no en un
archivo.

# ---- Servidor ----
NODE_ENV=development
PORT=3000                              # en Render la inyecta la plataforma: no la
cargues
FRONTEND_URL=http://localhost:5173     # en prod: la URL del Static Site, sin barra al
final (CORS)

# ---- Postgres local (docker compose) ----
POSTGRES_USER=postgres
POSTGRES_PASSWORD=tu_password_segura
POSTGRES_DB=libreria_db
POSTGRES_PORT=5432
DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:${POSTGRES_PORT}/${POS
TGRES_DB}
# en prod: la connection string directa de Neon
(postgresql://...@ep-....neon.tech/neondb?sslmode=require...)

# ---- Prisma: menos ruido en la consola ----
CHECKPOINT_DISABLE=1
PRISMA_HIDE_UPDATE_MESSAGE=1

# ---- Auth ----
JWT_SECRET=cambiame_por_una_cadena_larga_y_aleatoria   # en prod: generado con el botón
de Render, nunca este valor
```

frontend/.env.example

```
# Copiá este archivo a .env. El .env NO va a Git.
# Única variable del front. Queda EMBEBIDA en el build (cualquiera la ve con F12): nunca
un secreto acá.
#   local: http://localhost:3000/api
#   prod:  https://{{tu-back}}.onrender.com/api   ← se carga en el Static Site de
Render, no en este archivo
VITE_API_URL=http://localhost:3000/api
```